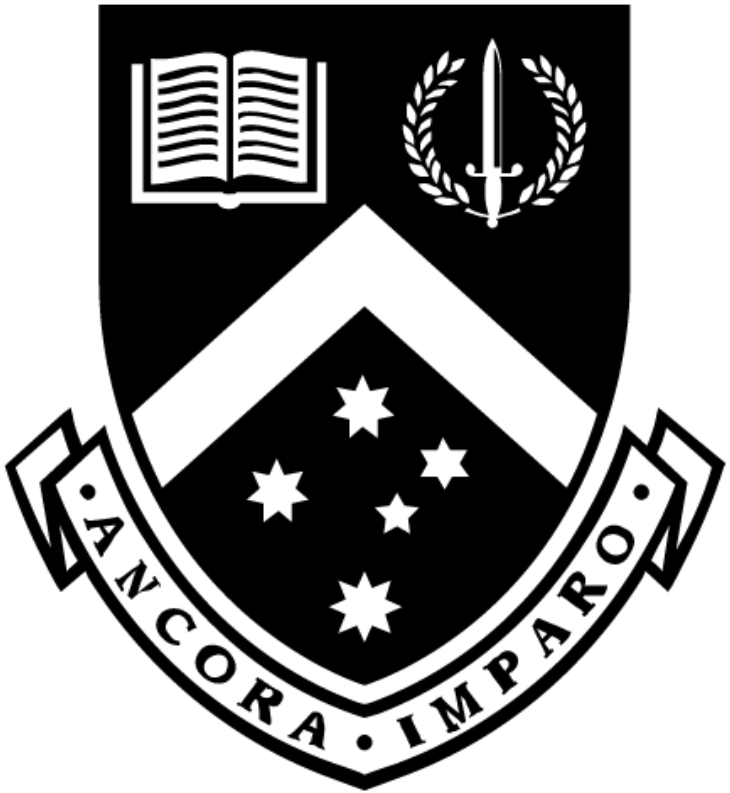
APAC HPC-AI COMPETITION

Team Monash DeepNeuron



nwchemgit/
nwchem

NWChem: Open Source High-Performance Computational Chemistry

62 Contributors

56 Issues

19 Discussions

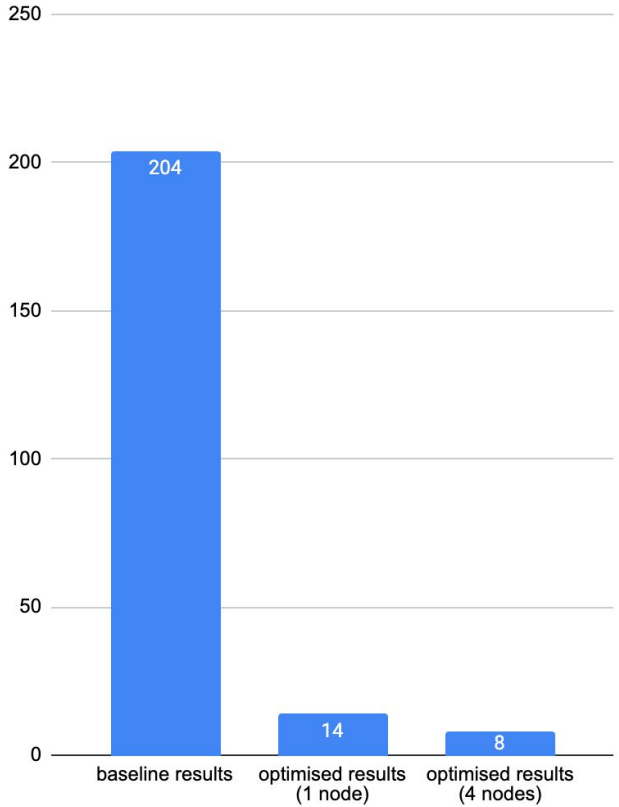
572 Stars

180 Forks

FINAL RESULTS

Compiler	intel -compiler-llvm/2025.2.0
Math Routine	intel- mkl/2025.2.0
ARMCI Network	MPI-PR
MPI	intel-mpi/2021.16

```
Optimised 4 node performance: 8~s
Optimised 1 node performance: 14~s
```



SCIENTIFIC METHODOLOGY

- Multivariate Testing vs inode quota limits
- Wrote scripts to reduce inode usage
- Data Collection vs KSU quota limits
- Conducted sampling via intel-mpi gtool flag

```
mpirun -np $TOTAL_RANKS \
        -ppn $RANKS_PER_NODE \
        -genv OMP_NUM_THREADS $OMP_NUM_THREADS \
        -genv OMP_PLACES cores \
        -genv OMP_PROC_BIND close \
        -gtool "vtune -collect hotspot -r vtunehot:0" \
        $APP $INPUT > $OUTPUT
```

```
$ find ./nwchem | wc
  31288   31288 1266694

$ ./clean.sh nwchem
...

$ find ./nwchem | wc
   2122    2122   83615
```

COMPILATION SETTINGS

```
# COMPILER FLAGS
export FCFLAGS="-Ofast -march=sapphirerapids -fp-model fast=2 -fopenmp \
    -i8 -fPIC -ipo -fallow-argument-mismatch \ -fno-operator-overloading-check"
export CCFLAGS="-Ofast -march=sapphirerapids -fp-model fast=2 -fPIC -ipo"
export CXXFLAGS="-Ofast -march=sapphirerapids -fp-model fast=2 -fPIC -ipo"

# ARMCI SETTINGS
export ARMCI_NETWORK=MPI-PR
export EXTERNAL_ARMCI_PATH=$NWCHEM_TOP/external-armci

# GLOBAL ARRAY CONFIGURATION
# export EXTERNAL_GA_PATH=$GA_INSTALL
```

CONVERGENCE DIFFERENCE

```
$ diff optimised_results.nwout baseline.nwout
35,39c35,39
<          Total DFT energy =     -917.738247953118
<       One electron energy =    -3287.432508164322
<            Coulomb energy =     1466.414001922894
<     Exchange-Corr. energy =     -112.235614418838
<  Nuclear repulsion energy =     1015.515872707148
---
>          Total DFT energy =     -917.738244976750
>       One electron energy =
>            Coulomb energy =
>     Exchange-Corr. energy =
>  Nuclear repulsion energy =     1015.516447774855
```

MPI DIRECTIVES

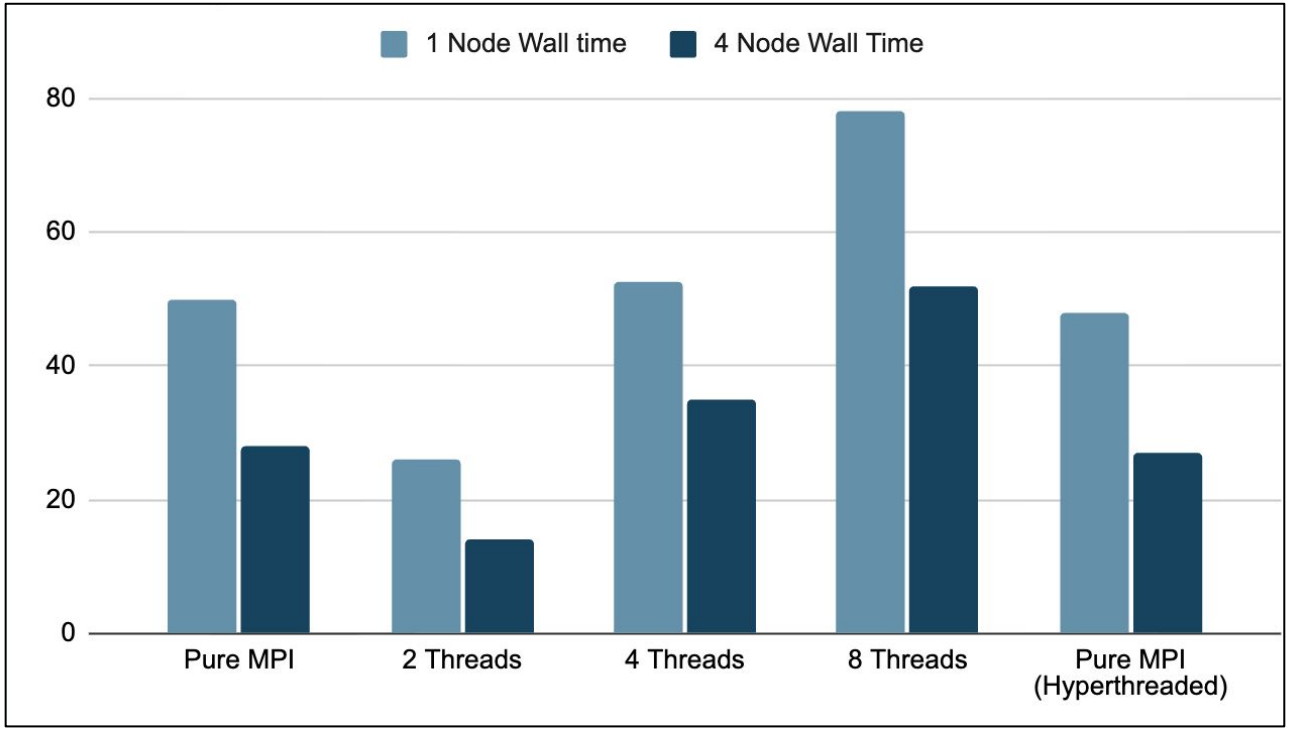
Multithreaded
open-mpi

```
mpirun -np $TOTAL_RANKS \
       --map-by ppr:$RANKS_PER_NODE:node:PE=$OMP_NUM_THREADS \
       --bind-to core \
       --report-bindings \
       $APP $INPUT > $OUTPUT
```

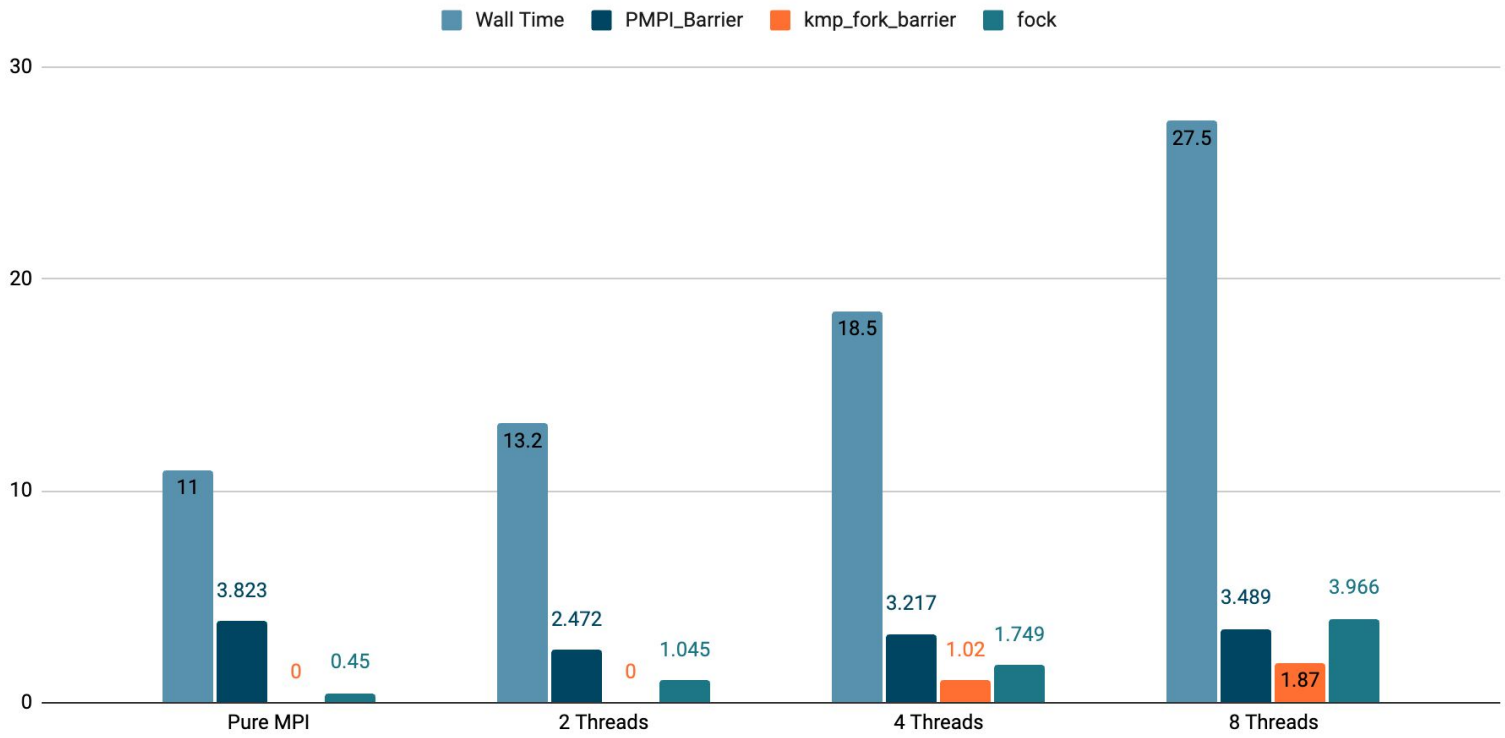
Pure MPI
intel-mpi

```
mpirun -np $TOTAL_RANKS \
       -ppn $RANKS_PER_NODE \
       -genv OMP_NUM_THREADS=$OMP_NUM_THREADS \
       -genv OMP_PLACES=cores \
       -genv OMP_PROC_BIND=close \
       -genv I_MPI_PIN=1 \
       -genv I_MPI_PIN_DOMAIN=core \
       -genv I_MPI_MAP_BY=node \
       $APP $INPUT > $OUTPUT
```

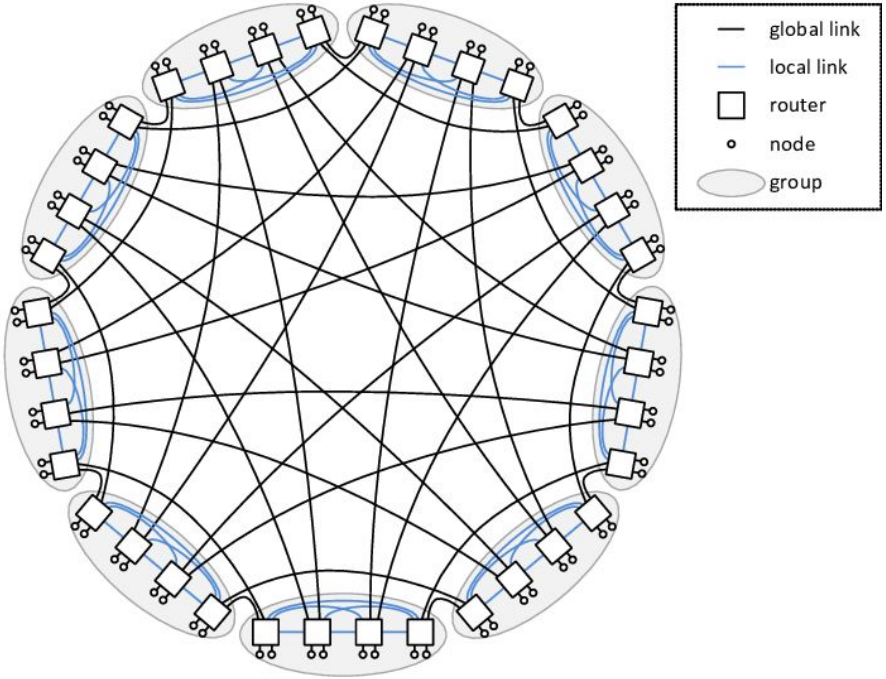
OPENMP MULTI/HYPER THREADING



INTEL-MPI THREADING ANALYSIS



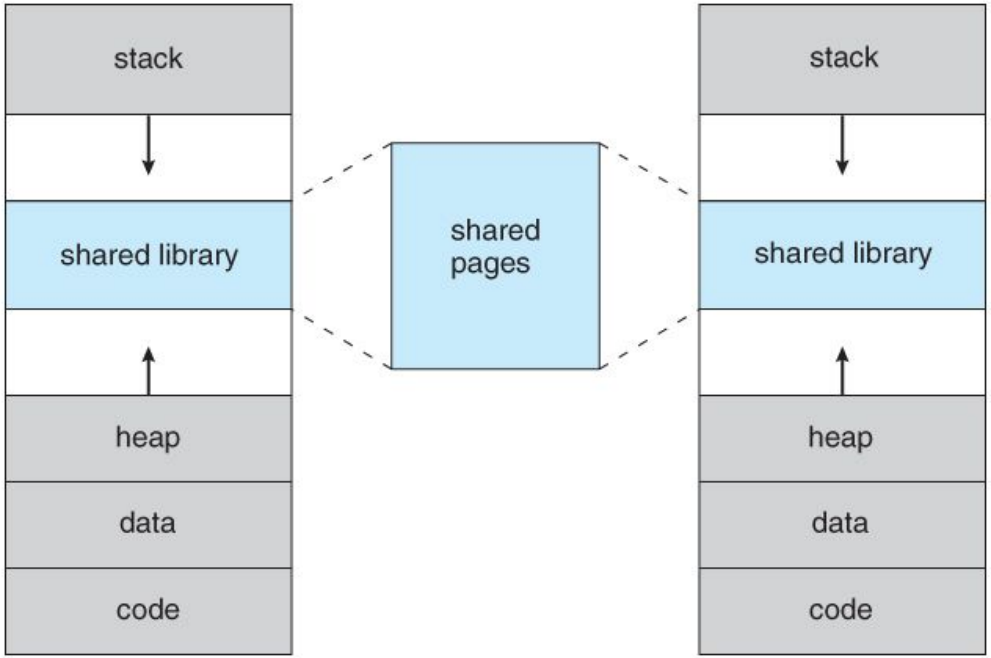
UCX (INFINIBAND)




```
export UCX_TLS=rc_x,dc_x,ud,self,sm
export UCX_NET_DEVICES=mlx5_0:1
export UCX_MEMTYPE_CACHE=n
export UCX_RNDV_SCHEME=get_zcopy
```

MEMORY ALLOCATION

`< memory stack 1100 mb heap 100 mb global 1100 mb`

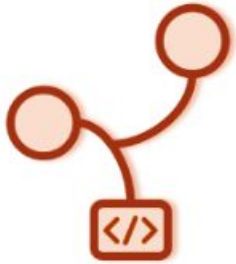


DFT/SCF CHANGES

```
$ diff optimised_results.nwout baseline.nwout
< memory stack 1100 mb heap 100 mb global 1100 mb
---
> memory stack 8000 mb heap 100 mb global 8000 mb noverify
6,7c7,8
< permanent_dir .
< scratch_dir /tmp
---
> permanent_dir .
> scratch_dir /tmp
51a53,66
>
53c68
<   semidirect
---
>   direct   # you can change this
```

sgl-project/**sglang**

SGLang is a fast serving framework for large language models and vision language models.



807 Contributors

528 Issues

290 Discussions

19k Stars

3k Forks

CUDA API VERSION

We found that through Conda we could download different Cuda API’s, and experiment with which was fastest. We found that **Cuda129** was the fastest, showing **7863.04** tokens per second on 2 H100 nodes compared to 5839 tokens per second on Cuda126 on 2 H100 nodes

This is likely because Sglang is able to use the newer cuBLAS and DeepGEMM kernels that are compatible with the newer SGLang versions.

DEEPGEMM

Using DeepGemm JIT compilation with SGLang will **improve throughput**, it's worth noting that DeepGemm JIT Compilation is **enabled by default** in SGLang so I haven't added any flags for it, though it will cause the first run of our scripts to take significantly longer (~10-20 mins) as the DeepGemm JIT kernels are pre-compiled.

Others have included scripts that are dedicated to performing this JIT DeepGemm kernel pre-compilation, but we have elected to allow SGLang to do it on the first run to remove unnecessary complexity when attempting to run our builds for the first time.

This error message implies you need to run the sglang.compile_deep_gemm command to pre-compile the kernels, this is not the case.

```
1: [2025-10-29 15:34:46 TP8] Entering DeepGEMM JIT Pre-Compile session. It may take a long time (typically 10-20 mins) if you have not run `sglang.compile_deep_gemm`. It is recommended to run `sglang.compile_deep_gem
m` with same args as `sglang.launch_server` for pre-compilation to reduce the overhead if you have not run it before. For example: `python3 -m sglang.compile_deep_gemm --model deepseek-ai/DeepSeek-V3 --tp 8 --trust-r
emote-code`
```

LAUNCHING SGLANG

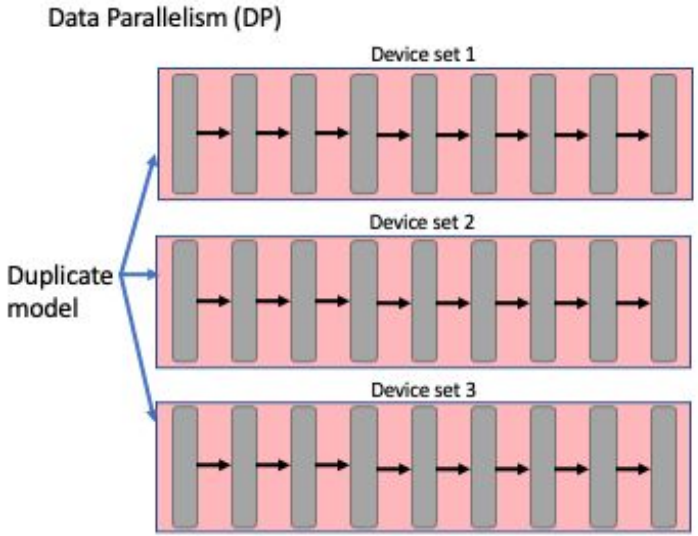
Python Venv Launch

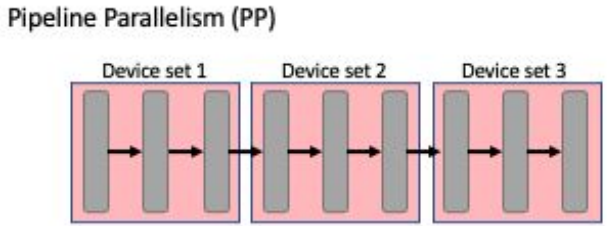
```
═══ Offline Throughput Benchmark Result ═══
Backend:                                 engine
Successful requests:                     2000
Benchmark duration (s):                  101.00
Total input tokens:                      626729
Total generated tokens:                  388685
Last generation throughput (tok/s):      71.95
Request throughput (req/s):              19.80
Input token throughput (tok/s):          6205.30
Output token throughput (tok/s):         3848.40
Total token throughput (tok/s):          10053.70
═══════════════════════════════════════════════
```

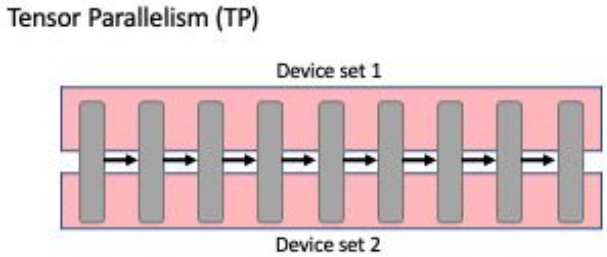
With Singularity

```
═══ Offline Throughput Benchmark Result ═══
Backend:                                 engine
Successful requests:                     2000
Benchmark duration (s):                  90.23
Total input tokens:                      626729
Total generated tokens:                  388685
Last generation throughput (tok/s):      73.18
Request throughput (req/s):              22.17
Input token throughput (tok/s):          6945.77
Output token throughput (tok/s):         4307.63
Total token throughput (tok/s):          11253.39
═══════════════════════════════════════════════
```

PARALLELISM TECHNIQUES







PARALLELISM WIN: BATCH SIZE

Experiments ran where the number of prompts was less than the 2000 showed a significant decline in offline throughput, for example our best results showed ~17k token's per second, if we were to half the number of prompts to 1000 prompts, we would see ~10k token's per second, a drop off of nearly half!

This made it clear that maximising the batch size of our program would maximise throughput, as this would ensure that the KV Cache remained saturated and that the GPU's parallelism capacities were maximised.

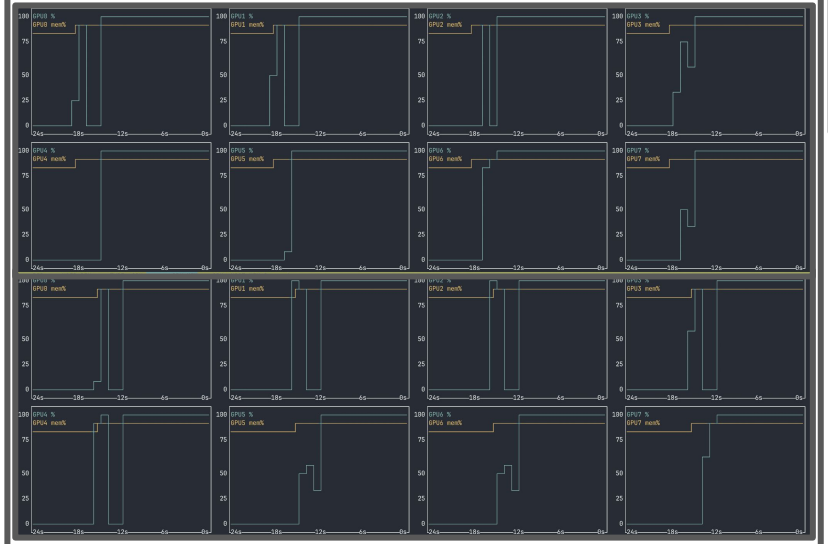
SINGLE NODE THROUGHPUT

We found that running deepseek R1 on one node **significantly reduced communication overhead**, such **that the advantages that can be achieved from using 2 nodes** (extra gpu memory and processing power) **where nearly trumped by the disadvantages of using 2 nodes** (primarily greater communication overhead). During our investigations into this issue, many pointed out that it may have been that we were **not fully utilising both nodes,** this led us to perform some profiling.

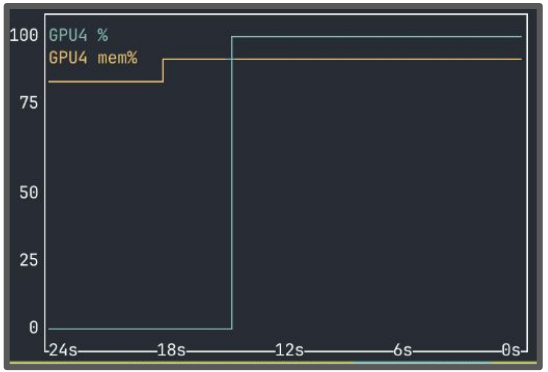
We found that profiling **SGLang's running at 2000 prompts was difficult**, but if we dropped the number of prompts be used then the performance was not comparable due to **different bottlenecks** arising to the full number of prompts. Profiling at 2000 prompts would lead to massive profiling files (~8gb per GPU), so we were left with the next best option which was monitoring GPU, CPU, and RAM utilization live, through tools like HTop and NVTop.

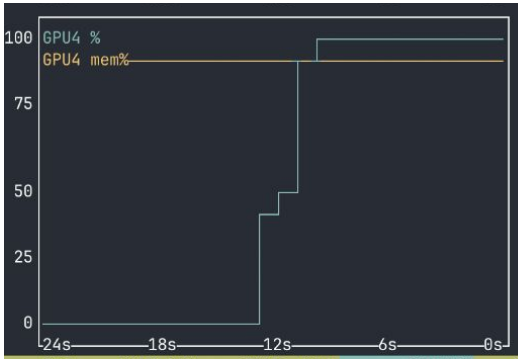
PROFILING RESULTS

2 Node Deployment, 11,381.11 Tok/Sec

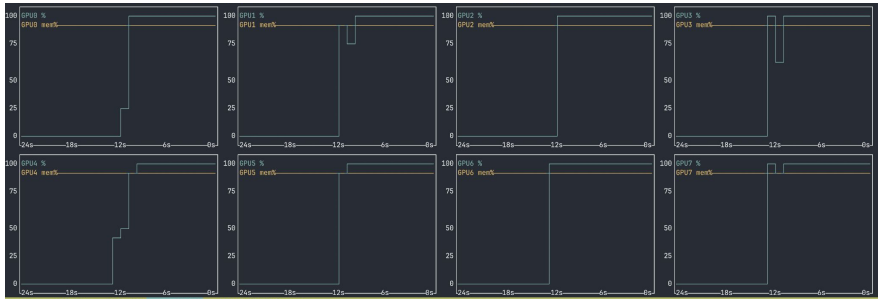


With baseline two node script as provided in run scripts (TP = 16, Direct python launch)





Single Node Deployment, 12,520.75 Tok/Sec



With TP = 8 and default python direct launch as iterated in provided run scripts.

PRE/POST TUNING

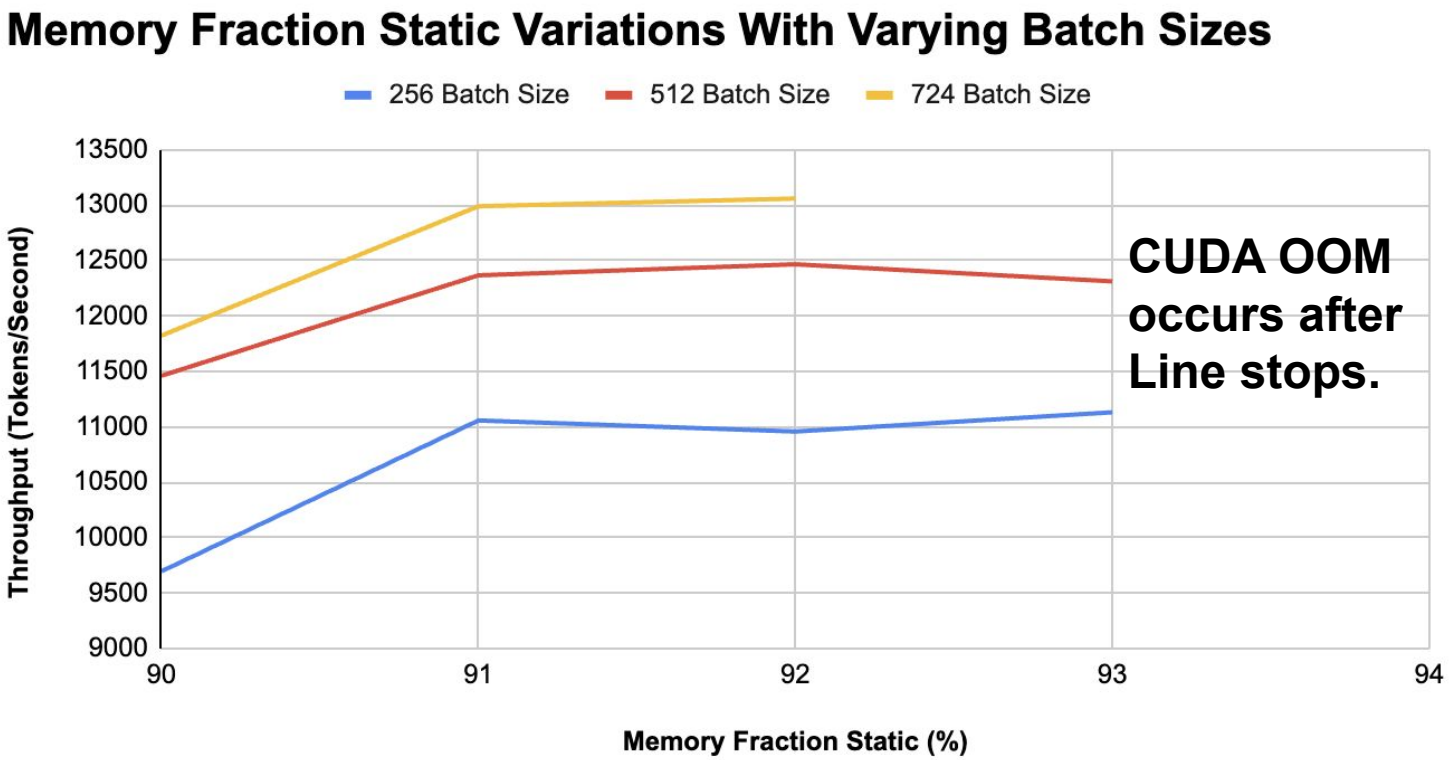
Before Tuning

GPU	MEM	CPU	HOST MEM	Command
135922MiB	95%	98%	2943MiB	sglang::scheduler_TP1
135922MiB	95%	95%	2953MiB	sglang::scheduler_TP2
135922MiB	95%	97%	2960MiB	sglang::scheduler_TP3
135922MiB	95%	95%	2959MiB	sglang::scheduler_TP4
135922MiB	95%	90%	2982MiB	sglang::scheduler_TP5
135922MiB	95%	95%	2951MiB	sglang::scheduler_TP6
135830MiB	94%	93%	2992MiB	sglang::scheduler_TP0
135442MiB	94%	90%	2966MiB	sglang::scheduler_TP7

After Tuning

GPU	MEM	CPU	HOST MEM	Command
142548MiB	99%	97%	2979MiB	sglang::scheduler_TP1
142548MiB	99%	99%	2998MiB	sglang::scheduler_TP2
142548MiB	99%	99%	3001MiB	sglang::scheduler_TP3
142548MiB	99%	99%	2979MiB	sglang::scheduler_TP4
142548MiB	99%	98%	2991MiB	sglang::scheduler_TP5
142548MiB	99%	100%	2989MiB	sglang::scheduler_TP6
142456MiB	99%	96%	2992MiB	sglang::scheduler_TP0
142068MiB	99%	98%	2989MiB	sglang::scheduler_TP7

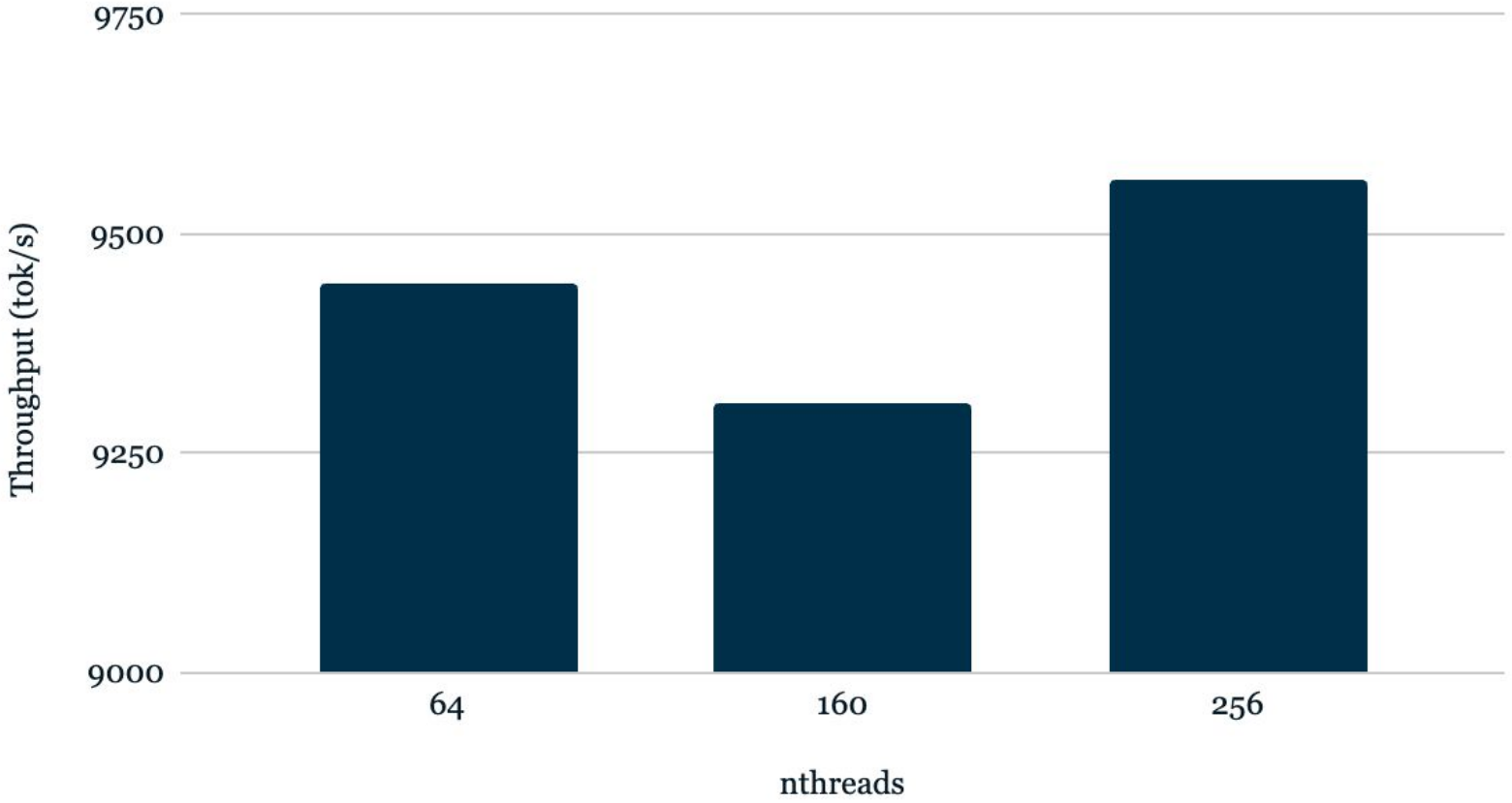
BATCH SIZE THROUGHPUT RESULTS



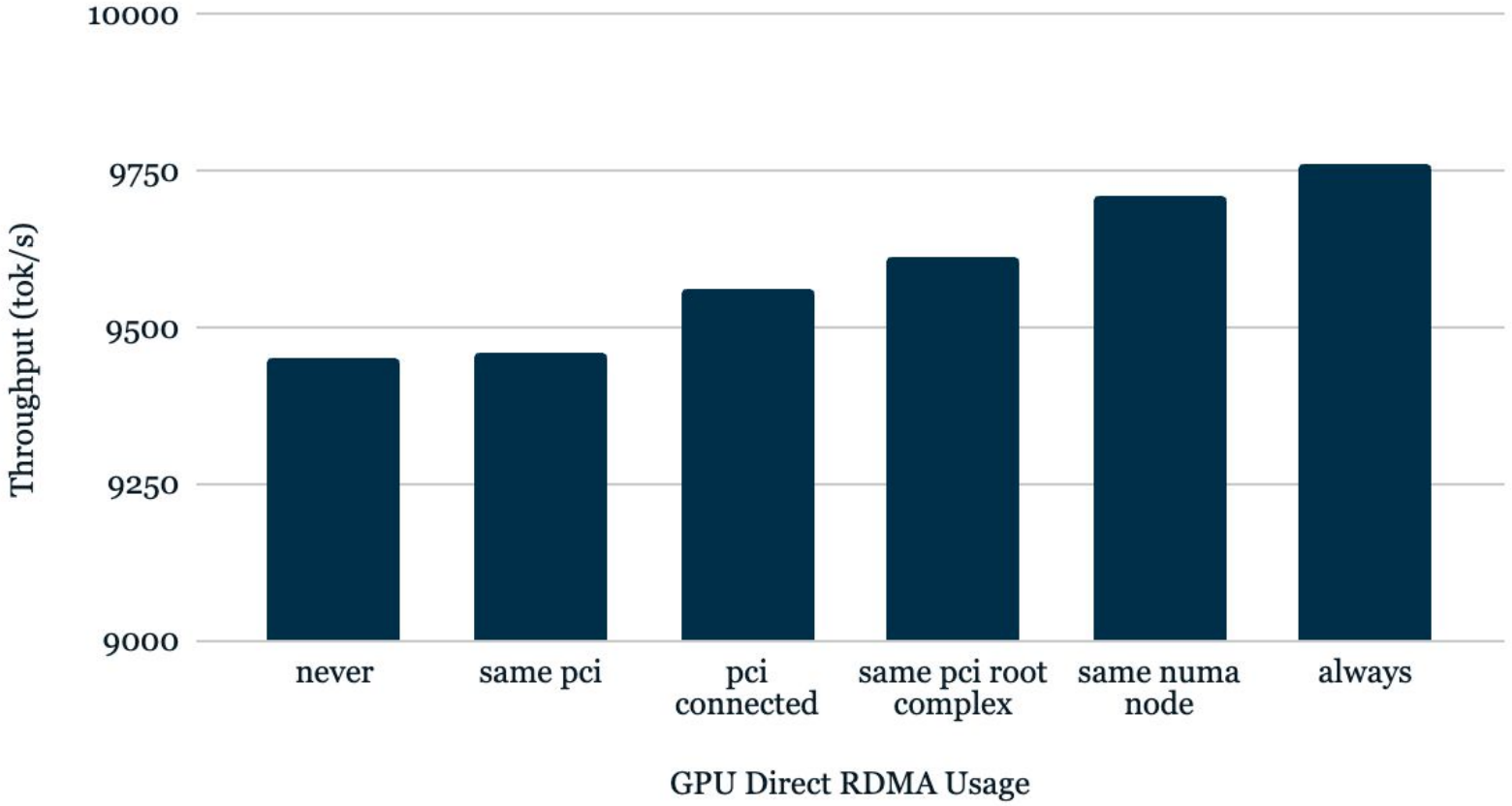
NETWORK TOPOLOGY

Nodes are connected through infiniband and RoCE v2, though Infiniband is primarily what we used. Internally each gpu is connected to each other through NVLink.
When attempting to optimise the network configuration for NCCL there are numerous parameters that can be optimised, we chose the ones we believed would make the largest performance improvements and experimented with those, on the next slide are our results from tuning those.

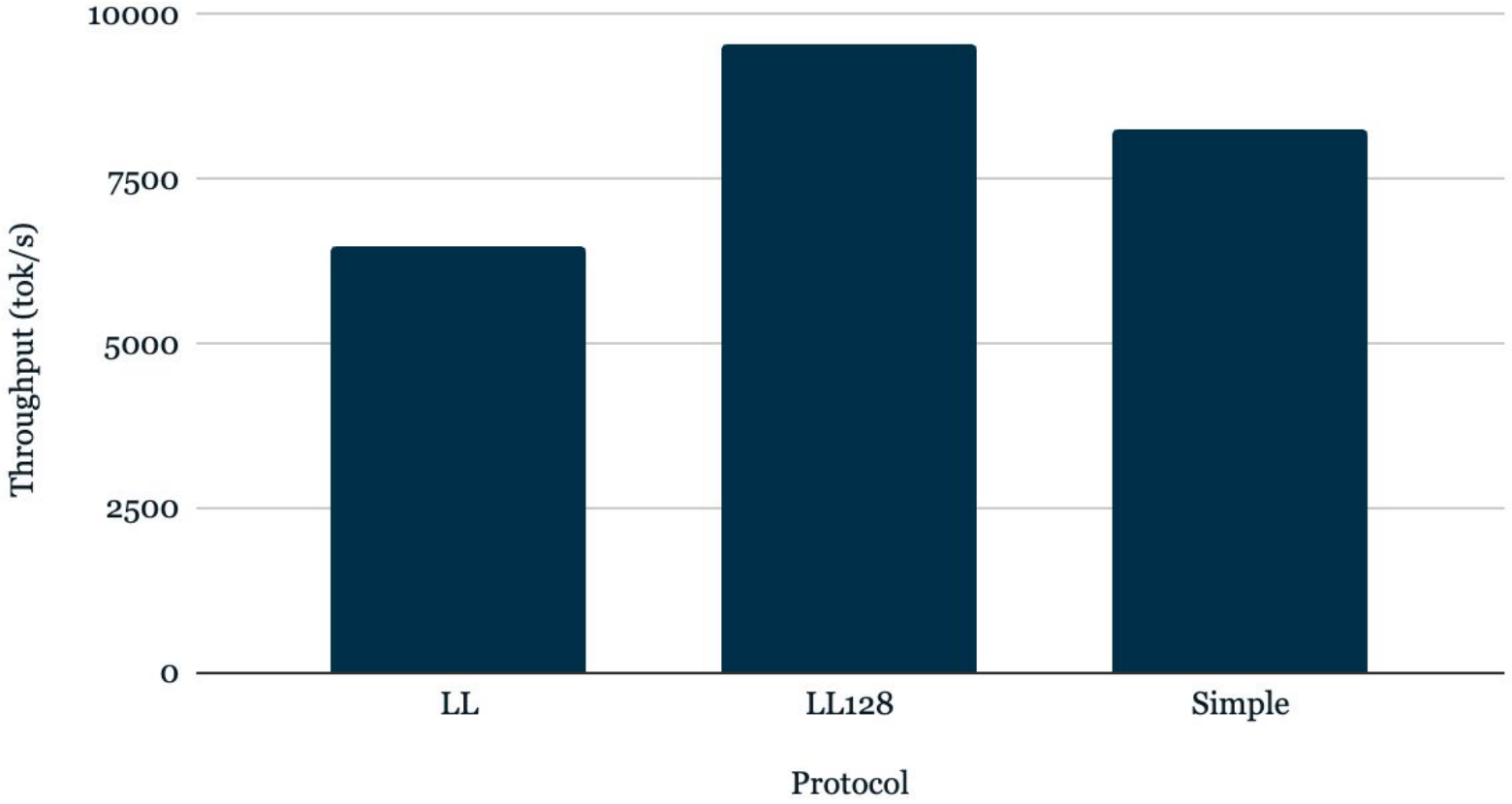
Throughput (tok/s) vs nthreads



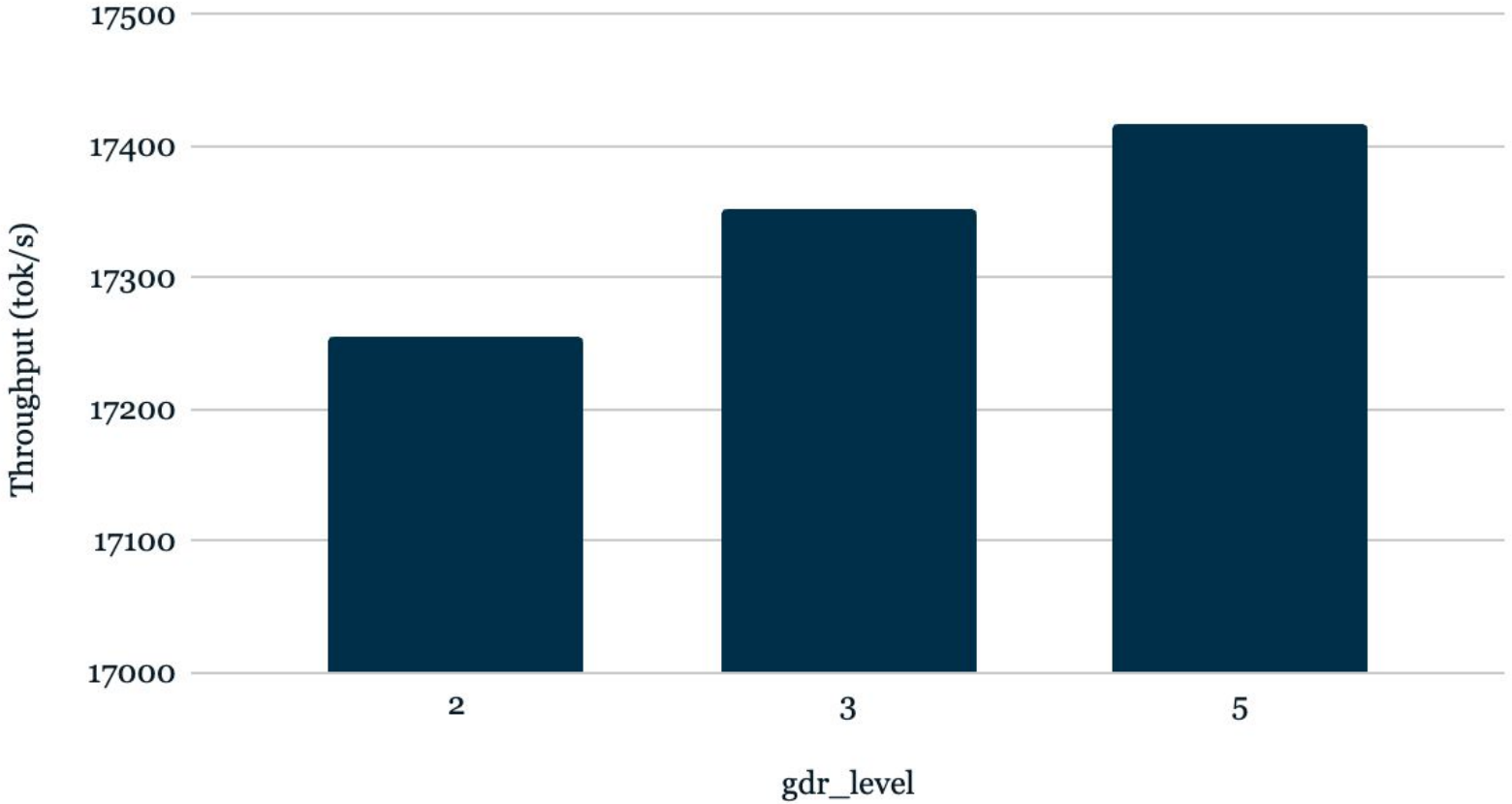
Throughput (tok/s) vs GPU Direct RDMA Usage



Throughput (tok/s) vs Protocol



Throughput (tok/s) vs gdr_level



PARALLELISATION TECHNIQUES CONT.

I stated earlier that Data Parallelism (DP) is king for throughput, however true DP across two nodes in sglang is not available in the form of an **offline** benchmark, however through online serving and benchmarking that we are able to perform true DP.

This has a large caveat though, it draws in many more required operations (load balancing, proper tokenization…) and most importantly **does not satisfy the criteria for this competition** to benchmark offline throughput.

I attempted it anyway, and found that the results were **~10k tokens per second**. In a real deployment scenario, we propose this as the most practical method for deploying DeepSeek R1.

FUTURE CONSIDERATIONS

TP	DP	PP	Throughput (tok/s)
4	4	2	17308.39
4	4	2	17218.03
4	1	2	12537.46
4	1	2	11436.32
2	2	4	16498.81
2	2	4	15228.52

Our current issue:

- Cross node deployment has **too much communication overhead**
- The Parallelism technique most adapt to fixing this is not present in SGLang **when performing offline benchmarks** across nodes

So we optimised for one node, we found that the optimal configuration was **TP = 4**, **DP = 4**, **PP = 2**, with **dp-attention**.
The TP == DP is a requirement for dp-attention, which speeds up the attention heads within the node, and allows for SGLang's shared experts (EP) optimisations.
The PP=2 helps reduce unnecessary communication within the node, and increases KV-Cache locality between the GPU's.

PARALLELISM ANALYSIS

+40% Throughput Increase from just parallelism configuration changes

+75% Throughput Improvement Over Provided Baseline

TP=8 (with Singularity and Other shown improvements)

```
====== Offline Throughput Benchmark Result =======
Backend:                                 engine
Successful requests:                     2000
Benchmark duration (s):                  81.79
Total input tokens:                      626729
Total generated tokens:                  388685
Last generation throughput (tok/s):      100.76
Request throughput (req/s):              24.45
Input token throughput (tok/s):          7662.45
Output token throughput (tok/s):         4752.10
Total token throughput (tok/s):          12414.55
==================================================
```

TP=4, DP=4, PP=2, DP-Attention

```
====== Offline Throughput Benchmark Result =======
Backend:                                 engine
Successful requests:                     2000
Benchmark duration (s):                  58.93
Total input tokens:                      626729
Total generated tokens:                  388685
Last generation throughput (tok/s):      52.44
Request throughput (req/s):              33.94
Input token throughput (tok/s):          10635.11
Output token throughput (tok/s):         6595.68
Total token throughput (tok/s):          17230.79
==================================================
```

FINAL RESULTS

+82% Throughput Improvement Over Provided Baseline

Baseline

```
====== Offline Throughput Benchmark Result =======
Backend:                                 engine
Successful requests:                     2000
Benchmark duration (s):                  105.66
Total input tokens:                      626729
Total generated tokens:                  388685
Last generation throughput (tok/s):      72.02
Request throughput (req/s):              18.93
Input token throughput (tok/s):          5931.79
Output token throughput (tok/s):         3678.78
Total token throughput (tok/s):          9610.57
================================================
```

One Node Best

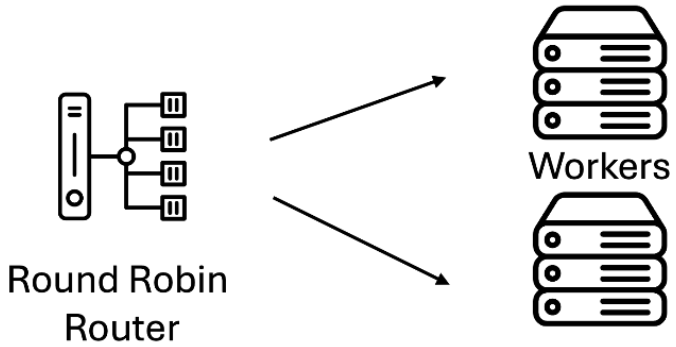
```
====== Offline Throughput Benchmark Result =======
Backend:                                 engine
Successful requests:                     2000
Benchmark duration (s):                  58.30
Total input tokens:                      626729
Total generated tokens:                  388685
Last generation throughput (tok/s):      54.24
Request throughput (req/s):              34.31
Input token throughput (tok/s):          10750.29
Output token throughput (tok/s):         6667.12
Total token throughput (tok/s):          17417.40
================================================
```

Two Node Best

```
====== Offline Throughput Benchmark Result =======
Backend:                                 engine
Successful requests:                     2000
Benchmark duration (s):                  59.61
Total input tokens:                      626729
Total generated tokens:                  388685
Last generation throughput (tok/s):      53.99
Request throughput (req/s):              33.55
Input token throughput (tok/s):          10513.61
Output token throughput (tok/s):         6520.33
Total token throughput (tok/s):          17033.94
================================================
```

ACTUALISED DEPLOYMENT

In a real world deployment, we propose using a router worker architecture, as shown **below**. In this scenario, each node would run an instance of a SGLang's DeepSeek worker.



ADDITIONAL CONSIDERATIONS

Here are some additional features/options we implemented or experimented with.

- **Torch Compile**
- **Nsight Profiling** with less prompts
- **Expert Parallelism** changes
- **Direct DeepGEMM pre-compilation** (no difference compared to current precompilation method)
- **Expert Parallelism** (implemented by default with current setup)
- **MTP** Multi Token Prediction
- **Hyperparameter Optimisation With NN** (too inefficient, uses too many resources)
- And many other things!

What do you call a chicken that’s good at maths?

A mathamachicken

Thanks For Listening!

Team Monash DeepNeuron